

Generics

COMPLETE LEARNING GUIDE

What You'll Learn

Generics let classes, interfaces, and methods operate on types specified by the caller at compile time, eliminating the unsafe, error-prone casting that pre-Java-5 code required whenever a collection held `Object` references. This guide covers why generics exist (type safety plus elimination of casts), how to write your own generic classes and methods, bounded type parameters that restrict what a type argument can be, the wildcard system (`?` extends, `?` super) and the PECS mnemonic that governs when to use each, and type erasure — the compiler mechanism that implements generics but also explains several of their more surprising limitations. Generics questions in interviews often center on wildcards, since `?` extends and `?` super are genuinely confusing without a clear mental model, and this guide gives you that model.

1. Why Generics Exist: Type Safety Without Casts

Before generics (pre-Java 5), a collection like `ArrayList` held `Object` references only, meaning every element retrieved required an explicit downcast, and a mismatched cast (adding a `String` to a list you intended for `Integers`) would compile fine and only fail with a `ClassCastException` at runtime, often far from the actual bug. Generics move that type check to compile time:

`List<String>` can only ever hold `Strings`, the compiler rejects an attempt to add an `Integer` at the call site, and no cast is needed when reading elements back out — the compiler inserts the cast for you invisibly, guaranteed safe because it already verified the type.

2. Writing a Generic Class

A generic class declares one or more type parameters in angle brackets after the class name (commonly single uppercase letters like `T` for "Type," `E` for "Element," `K/V` for "Key/Value") which then act as placeholder types usable anywhere inside the class body — fields, method parameters, return types. The type argument is supplied by the caller when instantiating the class (`Box<String> box = new Box<>();`), and from that point forward the compiler enforces that only `Strings` can go in or come out of that specific `box` instance.

JAVA

```
public class Box<T> {
    private T contents;

    public void set(T contents) { this.contents = contents; }
    public T get() { return contents; }
}

Box<String> stringBox = new Box<>();
stringBox.set("hello");
String value = stringBox.get();    // no cast needed -- compiler already knows
//StringBox.set(42);               // compile error -- caught before runtime
```

3. Generic Methods and Bounded Type Parameters

A single method can be generic even inside a non-generic class, by declaring its own type parameter before the return type (public static <T> T firstElement(List<T> list)). Bounded type parameters restrict what concrete types are legal arguments using extends (which, for generics, means "extends or implements") — <T extends Comparable<T>> restricts T to types that can be compared to themselves, which is exactly what's needed to write a generic max() function that calls compareTo() on the elements.

JAVA

```
public static <T extends Comparable<T>> T max(List<T> list) {
    T best = list.get(0);
    for (T item : list) {
        if (item.compareTo(best) > 0) best = item;
    }
    return best;
}
// works for List<Integer>, List<String>, or any Comparable type
```

4. Wildcards: ? extends and ? super

A wildcard represents an unknown type argument when you don't need to name it, used to write methods that accept a family of generic types rather than one exact instantiation. List<? extends Number> can refer to a List<Integer>, List<Double>, or List<Number> — you can safely read elements out as Number, but you cannot add anything to it (except null), because the compiler can't guarantee which specific subtype is actually backing the list at runtime. List<? super Integer> can refer to a List<Integer>, List<Number>, or List<Object> — you can safely add Integers into it, but reading elements out only guarantees an Object, since the actual list could be storing a wider type.

- ? extends T ("upper bounded") — safe to READ as T, unsafe to WRITE (except null). Use for producers.
- ? super T ("lower bounded") — safe to WRITE T (or its subtypes), reads only guarantee Object. Use for consumers.

- PECS mnemonic: "Producer Extends, Consumer Super" — if a parameter produces T values you'll read, use extends; if it consumes T values you'll write, use super.
- A plain, unbounded List<?> supports neither adding (except null) nor any typed reading beyond Object.

5. PECS in Practice: Copying Between Collections

The classic real-world PECS example is a copy method: the source list is a producer (you read T out of it, so it should be List<? extends T>) and the destination list is a consumer (you write T into it, so it should be List<? super T>). This lets the method correctly accept `copy(List<Integer> source, List<Number> dest)` as well as `copy(List<Integer> source, List<Object> dest)`, which a naively-typed `copy(List<T> source, List<T> dest)` signature would incorrectly reject.

JAVA

```
public static <T> void copy(List<? extends T> source, List<? super T> dest) {  
    for (T item : source) {  
        dest.add(item);  
    }  
}  
// works for copy(List<Integer>, List<Number>) and copy(List<Integer>,  
List<Object>)
```

6. Type Erasure and Its Consequences

The JVM has no concept of generics at runtime — the compiler "erases" all generic type information after compile-time checking, replacing type parameters with their bound (Object if unbounded) and inserting the necessary casts, so that a List<String> and a List<Integer> are, at the bytecode level, both simply List. This is why you cannot create an array of a generic type (new T[10] is illegal), cannot use instanceof with a parameterized type (obj instanceof List<String> is illegal — only obj instanceof List<?> compiles), and cannot overload two methods that would erase to the identical signature (void process(List<String> list) and void process(List<Integer> list) in the same class is a compile error, since both erase to process(List)).

Common Pitfalls

- Trying to call list.add(...) on a List<? extends T> reference and being confused by the compile error — extends wildcards are read-only by design.
- Writing new T[10] inside a generic class and hitting a compile error, not realizing type erasure removes T entirely at runtime, so the JVM has no idea what array type to allocate.
- Attempting obj instanceof List<String> and getting a compile error — only the unbounded obj instanceof List<?> is legal due to erasure.
- Confusing which wildcard to use — forgetting PECS (Producer Extends, Consumer Super) and defaulting to plain T everywhere, then hitting unnecessary compile errors on valid call sites.

- Assuming two overloaded methods differing only in generic type parameter (`List<String>` vs `List<Integer>`) are legal overloads — they erase to the same signature and won't compile.
- Using a raw type (`List` instead of `List<T>`) to "avoid generics complexity," losing all compile-time type checking and reintroducing the `ClassCastException` risk generics were built to prevent.

Key Takeaways

- Generics move type-checking from runtime (`ClassCastException`) to compile time, eliminating manual casts in the process.
- Bounded type parameters (`<T extends Comparable<T>>`) restrict which types a generic method or class accepts, enabling operations like `compareTo()` to be called safely.
- PECS: use `? extends T` for a producer you only read from, `? super T` for a consumer you only write to.
- Type erasure means generic type information disappears at runtime — no generic arrays, no instanceof with a parameterized type, no overloads that erase identically.
- Never use raw types (`List` instead of `List<T>`) — doing so discards all of generics' compile-time safety.

Learn More

Oracle Java Tutorials — Generics

<https://docs.oracle.com/javase/tutorial/java/generics/index.html>

Oracle Java Tutorials — Wildcards

<https://docs.oracle.com/javase/tutorial/java/generics/wildcards.html>

Baeldung — Java Generics

<https://www.baeldung.com/java-generics>